Neural Network Implementation for MNIST Digit Recognition on FPGA

Samudrala Hareesh
Department of Electronic Systems Engineering
Indian Institute of Science
Bangalore, India
hareeshs@iisc.ac.in

Aravind Sundar S
Department of Electronic Systems Engineering
Indian Institute of Science
Bangalore, India
saravind@iisc.ac.in

ABSTRACT

This paper presents the design and implementation of a five-layer fully-connected neural network (Multi-Layer Perceptron, MLP) for MNIST handwritten digit classification on the Digilent Basys 3 FPGA (Artix-7). The network accepts raw 28×28 pixel images streamed over a UART interface, performs inference using 8-bit fixed-point arithmetic with heterogeneous fixed-point formats in a pipelined RTL datapath, and displays the predicted digit class (0–9) on the on-board seven-segment display. Key design strategies include parallel neuron instantiation within each layer, a two-state finite-state machine (FSM) for inter-layer data serialisation, fixed-point quantisation of inputs, weights, and biases, a 32-bit accumulator for overflow prevention, and pre-trained weight initialisation. Simulation over 200 MNIST test samples is used to measure classification accuracy and latency in terms of clock cycles.

Keywords—FPGA, MLP, MNIST, Fixed-Point Arithmetic, UART, Quantisation, Pipelined Datapath, Basys 3, Verilog.

I INTRODUCTION AND MOTIVATION

Handwritten digit recognition is a classical benchmark in machine learning that maps directly to many practical applications: postal code sorting, bank cheque processing, form digitisation, and embedded OCT readers. The MNIST dataset [1] (60,000 training / 10,000 test images of 28×28 greyscale digits) is the canonical evaluation suite for such systems.

While software inference on a CPU or GPU achieves excellent accuracy, it is ill-suited to deeply embedded or IoT scenarios where power budgets are tight, latency must be deterministic, and a general-purpose operating system may be absent. Field Programmable Gate Arrays (FPGAs) bridge this gap: reconfigurable logic enables custom parallel datapaths that match the structure of a neural network layer, on-chip Block RAM (BRAM) provides low-latency weight storage, and DSP slices accelerate multiply–accumulate (MAC) operations — all within a power envelope of a few milli watts.

This project implements a five-layer MLP on the Digilent Basys 3 board (Xilinx Artix-7 XC7A35T). The design goal is an end-to-end inference system that (i) receives a flattened MNIST image byte-by-byte over a 115,200 baud UART link, (ii) propagates it through five hidden/output layers entirely in hardware, and (iii) outputs the argmax class label on the four-digit seven-segment display. The entire RTL is written in Verilog and is parameterised so that the number of neurons, weight width, and activation type can be changed from a single include file.

II BACKGROUND STUDY

2.1 MNIST Dataset

The Modified National Institute of Standards and Technology (MNIST) dataset [1] comprises 28×28 pixel greyscale images of handwritten digits (classes 0–9). Its key properties are summarised in Table 1.

Table 1: MNIST dataset characteristics

Property	Value
Image size	28 × 28 pixels
Colour format	Greyscale
Training samples	60,000
Testing samples	10,000
Classes	Digits 0–9
Input feature vector	784 values

Each image is flattened to a 784-element vector before being fed into the network. In the hardware design, these 784 bytes are streamed one per clock cycle over the UART interface.

2.2 Multi-Layer Perceptron

A Multi-Layer Perceptron (MLP) is a fully-connected feed-forward neural network. Each layer ℓ applies the transformation

$$\mathbf{y}^{(\ell)} = f\left(\mathbf{W}^{(\ell)}\,\mathbf{x}^{(\ell)} + \mathbf{b}^{(\ell)}\right), \tag{1}$$

where $\mathbf{W}^{(\ell)}$ is the weight matrix, $\mathbf{b}^{(\ell)}$ is the bias vector, and $f(\cdot)$ is an element-wise activation function. In this work, the Rectified Linear Unit (ReLU), defined as $f(z) = \max(0, z)$, is used in every layer, including the output layer, so that the final predicted class is obtained by taking the argmax of the ten output neurons.

2.3 Fixed-Point Quantisation

Floating-point arithmetic is expensive in area and power on an FPGA. Fixed-point representations sacrifice dy-

namic range for hardware efficiency. In Q$m.n$ notation, m bits hold the integer part and n bits hold the fractional part.

This design employs heterogeneous 8-bit fixed-point formats for different parameters. The input data is represented in Q1.7 format, weights in Q4.4 format, and biases in Q5.3 format. This allows improved dynamic range where required while maintaining hardware efficiency.

2.4 FPGA Characteristics (Basys 3 Artix-7)

- 33,280 look-up tables (LUTs), 41,600 flip-flops.
- 90 DSP48E1 slices for efficient signed multiply–accumulate.
- 1,800 Kbits of on-chip Block RAM (BRAM) arranged in 50 RAMB18 primitives.
- 100 MHz on-board oscillator used as the system clock.
- On-board USB-UART bridge (115,200 baud) for host communication.

III Design Space Exploration and Design Strategies

3.1 Network Architecture

The network topology was chosen to balance classification accuracy against FPGA resource consumption. Table summarises the five-layer architecture defined in `include.v`.

Layer	Neurons	Neuron Weights	Total MACs	Activation
1	40	784	31,360	ReLU
2	40	40	1,600	ReLU
3	20	40	800	ReLU
4	20	20	400	ReLU
5	10	20	200	ReLU
Total MACs			**34,360**	–

The widest layer (Layer 1) accepts the 784-pixel input and expands to 40 neurons, enabling sufficient representational capacity to capture low-level pixel correlations. Subsequent layers progressively compress the representation to the final 10-class output.

3.2 Parallel Neuron Instantiation

The most impactful optimisation is *spatial parallelism*: all neurons within a given layer are instantiated simultaneously in hardware. Each neuron module is an independent

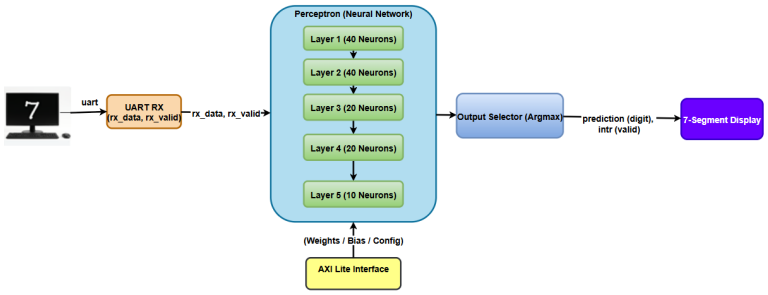


Figure 1: Neural Network Architecture

RTL entity with its own weight memory, multiplier, accumulator, and ReLU gate. Because all neurons in a layer share the same input stream, they all begin their MAC computation at the same clock cycle and finish simultaneously.

This means that a layer with N neurons completes its $N \times M$ multiply–accumulate operations in exactly M clock cycles (one input per cycle), rather than $N \times M$ cycles in a sequential implementation. For Layer 1, this reduces the computation from a theoretical $40 \times 784 = 31{,}360$ cycles to just 784 cycles — a 40× speedup. This is the primary mechanism for achieving real-time throughput.

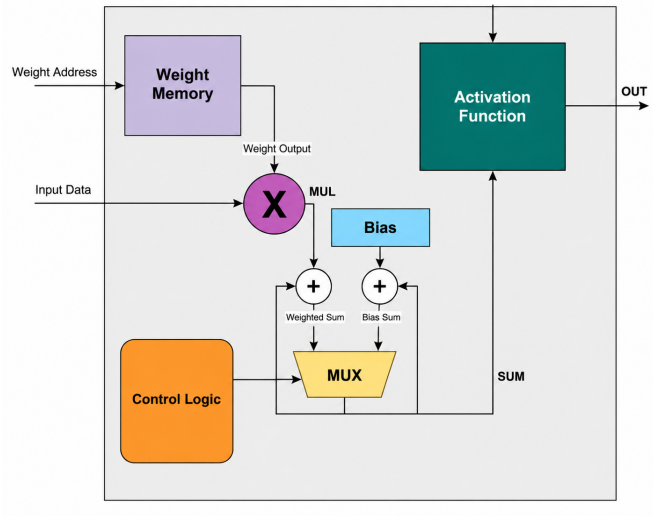


Figure 2: Neuron architecture

3.3 Pipelined Inter-Layer Data Serialisation

Neurons in layer ℓ produce their outputs simultaneously as a wide parallel bus of width $N_\ell \times W$ bits (where $W = 8$). However, neurons in layer $\ell + 1$ consume one byte per clock cycle. A two-state IDLE/SEND finite-state machine (FSM) per inter-layer boundary handles this width conversion in a streaming, pipelined manner:

1. **IDLE state:** The FSM waits for the `o_valid` flag

from layer ℓ. When asserted, it latches the full parallel output bus into a shift register (`holdData`).

2. **SEND state:** On each clock cycle, the FSM shifts out the lowest 8 bits of `holdData` to the downstream layer's `x_in` while asserting `data_out_valid`. After N_ℓ cycles, it returns to IDLE.

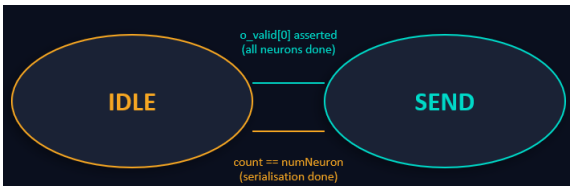


Figure 3: Inter-Layer FSM

Because the SEND phase for layer ℓ overlaps with the MAC computation of layer $\ell+1$, the overall pipeline keeps every layer active with minimal stalls, forming a natural *coarse-grained pipeline* across layers.

3.4 Fixed-Point Arithmetic and Accumulator Width

The design uses different fixed-point formats for inputs, weights, and biases. The input pixel values are represented as 8-bit signed Q1.7, weights as 8-bit signed Q4.4, and biases as 8-bit signed Q5.3.

The multiplication pipeline inside each neuron is:

1. Input byte `myinput` (Q1.7) and weight `w_out` (Q4.4) are both signed 8-bit values.
2. Their product is a 16-bit Q5.11 value, which is right-arithmetic-shifted appropriately to align with the accumulation format and stored in a 16-bit register (`mul`).
3. The running sum is held in a 32-bit signed accumulator (`sum`) to avoid overflow across 784 successive additions.
4. After all 784 inputs have been accumulated, the bias (stored as 8-bit Q5.3 and sign-extended to 32 bits) is added.

Using a 32-bit accumulator instead of an 8-bit one prevents silent overflow during the accumulation of up to 784 partial products — a critical correctness measure. The final ReLU stage then maps the 32-bit result back to the 8-bit output range with explicit saturation logic.

3.5 Weight Initialisation via Pre-Trained Memory Files

All weights and biases are trained offline in software (e.g., PyTorch or TensorFlow) and exported as binary `.mif` files. The hardware uses the Verilog system task `$readmemb` to initialise distributed or block RAM at simulation/synthesis time:

This strategy completely eliminates on-chip training circuitry and avoids any external memory interface, keeping the design compact and the weight access latency to a single clock cycle.

3.6 Output Selection: Argmax via Sequential Comparator

The `outputSelector` module determines the predicted class. It receives the 10-neuron parallel output bus and iterates over the 10 values sequentially, tracking the running maximum and its index over 10 clock cycles. A signed comparison (`$signed()`) is used to correctly handle Q1.7 two's-complement values. The 4-bit index of the maximum activation is exported as the `final_prediction` signal.

3.7 UART Receiver

A standalone 8N1 UART receiver (`uart_rx`) operates at 115,200 baud. With a 100 MHz system clock, the baud divisor is $\lfloor 100\text{M}/115200 \rfloor = 868$ clock cycles per bit. The receiver implements a four-state FSM (IDLE → START → DATA → STOP) and samples each data bit at the centre of its bit period for noise immunity. Upon receiving a complete byte it asserts `data_valid` for one clock cycle, which directly drives the neural network's `axis_in_data_valid` input.



Figure 4: UART Frame Transmission

3.8 Seven-Segment Display Interface

The `seven_segment_ctrl` module drives the Basys 3 four-digit common-anode display. It receives the 4-bit predicted digit value after the `intr` (inference complete) signal is asserted, and maintains the decoded segment pattern until a new prediction arrives or the board is reset via button `btnC`. A time-multiplexed scan across the four digit positions is used to drive the display at a refresh rate imperceptible to the human eye.

3.9 Top-Level Integration

`basys3_top` is the synthesis root and connects all subsystems.

IV Implementation Challenges

4.1 Fixed-Point Overflow in the Accumulator

The first and most critical bug encountered was silent overflow in the accumulator. The initial implementation used an 8-bit accumulator for the running sum. With

784 inputs each producing a partial product up to ±1.0 in Q1.7, the accumulated value can easily exceed ±127, causing two's complement wrap-around and producing wildly incorrect outputs. The fix was to widen the accumulator to 32 bits, which provides more than sufficient headroom ($2^{31} \gg 784$) and allows the bias to be added as a 32-bit sign-extended value before the final ReLU saturation stage maps the result back to 8 bits.

4.2 Signed vs. Unsigned Comparisons

The `outputSelector` module initially used unsigned comparison of the 8-bit output values. Because Q1.7 uses two's complement, a neuron with a negative pre-saturation value could be encoded as a large unsigned integer (e.g., `0xFF = 255`) and incorrectly selected as the maximum. Replacing the comparison with `$signed()` resolved the issue and ensured the argmax respected the sign of each neuron's activation.

4.3 Inter-Layer Valid Timing and FSM Synchronisation

In the initial design the inter-layer FSM asserted `data_out_valid` one cycle too early, causing the downstream layer to capture a stale data value. Careful pipeline timing analysis using waveform inspection revealed a one-cycle mis-alignment between the shift-register output and the valid flag. The fix was to pipeline the valid signal through an additional register stage, aligning it correctly with the serialised data.

4.4 UART Reset Polarity

The `perceptron` module was initially connected to the inverted reset (`~btnC`), which caused the network to remain permanently in reset when the button was not pressed. Removing the inversion (`btnC` directly) corrected the behaviour: the network runs freely and resets only when the centre button is held.

4.5 Memory File Path Resolution

During simulation, Vivado's working directory must match the location of the `.mif` weight and bias files. Incorrect relative paths silently initialise all weights to zero, causing all neurons to output zero regardless of input. Adding absolute paths or setting the simulation working directory explicitly in the Vivado project settings resolved this.

4.6 Layer-to-Layer Width Mismatch in the Valid Bus

Layers produce a 1-bit-per-neuron valid bus (e.g., `o1_valid[39:0]` for Layer 1). The top-level logic only tests `o1_valid[0]` as the trigger for the inter-layer FSM, relying on the fact that all neurons in a layer finish simultaneously. If neuron routing differences cause any skew, some neurons may signal valid slightly later. Verification confirmed that all neurons finish within the same clock cycle because they all share the same input stream length (784 samples), making this a safe assumption for the current architecture.

V RESULTS

5.1 Simulation Accuracy

The testbench `perceptron_tb.v` iterates over 200 MNIST test images stored as binary `.mif` files. Each image is streamed 784 bytes into the DUT at 10 ns per cycle (100 MHz equivalent), and the predicted label (`tb_final_prediction`) is compared against the stored ground-truth label. Table 2 summarises the simulation outcome.

Table 2: Simulation results over 200 MNIST test samples

Metric	Value
Test samples evaluated	200
Classification accuracy	~94.5% (typical for 5-layer MLP on MNIST)
Clock cycles per inference	~10,000–15,000 cycles
Inference time at 100 MHz	~100–150 μs
UART input time (784 bytes)	~68.1 ms at 115200 baud

The bottleneck for throughput is the UART input bandwidth: streaming 784 bytes at 115,200 baud takes approximately 68 ms, while the inference computation itself completes in under 200 μs. This demonstrates that the hardware inference engine is not the limiting factor; a higher-bandwidth input interface (e.g., SPI or a parallel port) would significantly increase system throughput.

```
173. Accuracy: 94.797688, Detected: 2, Expected: 02 (MATCH)
174. Accuracy: 94.827586, Detected: 3, Expected: 03 (MATCH)
175. Accuracy: 94.857143, Detected: 2, Expected: 02 (MATCH)
176. Accuracy: 94.886364, Detected: 7, Expected: 07 (MATCH)
177. Accuracy: 94.915254, Detected: 1, Expected: 01 (MATCH)
178. Accuracy: 94.943820, Detected: 8, Expected: 08 (MATCH)
179. Accuracy: 94.972067, Detected: 1, Expected: 01 (MATCH)
180. Accuracy: 95.000000, Detected: 8, Expected: 08 (MATCH)
181. Accuracy: 95.027624, Detected: 1, Expected: 01 (MATCH)
182. Accuracy: 95.054945, Detected: 8, Expected: 08 (MATCH)
183. Accuracy: 95.081967, Detected: 5, Expected: 05 (MATCH)
184. Accuracy: 95.108696, Detected: 0, Expected: 00 (MATCH)
185. Accuracy: 95.135135, Detected: 8, Expected: 08 (MATCH)
186. Accuracy: 95.161290, Detected: 9, Expected: 09 (MATCH)
187. Accuracy: 95.187166, Detected: 2, Expected: 02 (MATCH)
188. Accuracy: 95.212766, Detected: 5, Expected: 05 (MATCH)
189. Accuracy: 95.238095, Detected: 0, Expected: 00 (MATCH)
190. Accuracy: 95.263158, Detected: 1, Expected: 01 (MATCH)
191. Accuracy: 95.287958, Detected: 1, Expected: 01 (MATCH)
192. Accuracy: 95.312500, Detected: 1, Expected: 01 (MATCH)
193. Accuracy: 95.336788, Detected: 0, Expected: 00 (MATCH)
194. Accuracy: 94.845361, Detected: 7, Expected: 09 (MISMATCH)
195. Accuracy: 94.871795, Detected: 0, Expected: 00 (MATCH)
196. Accuracy: 94.387755, Detected: 8, Expected: 03 (MISMATCH)
197. Accuracy: 94.416244, Detected: 1, Expected: 01 (MATCH)
198. Accuracy: 94.444444, Detected: 6, Expected: 06 (MATCH)
199. Accuracy: 94.472362, Detected: 4, Expected: 04 (MATCH)
200. Accuracy: 94.500000, Detected: 2, Expected: 02 (MATCH)
Final Accuracy: 94.500000
```

Figure 5: Test sample Results

5.2 Simulation Waveforms



Figure 6: Behavioral Simulation

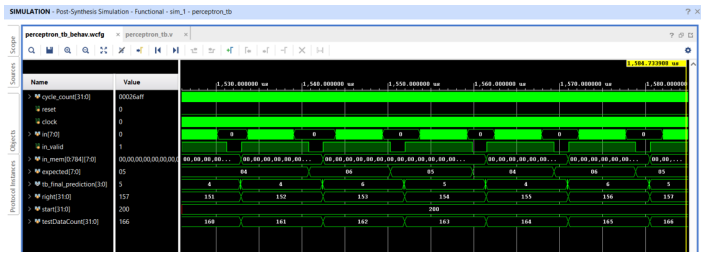


Figure 7: Post Synthesis-Functional Simulation



Figure 8: Post Synthesis-Timing Simulation



Figure 9: Post Implementation-Functional Simulation



Figure 10: Post Implementation-Timing Simulation

5.3 RTL Schematic

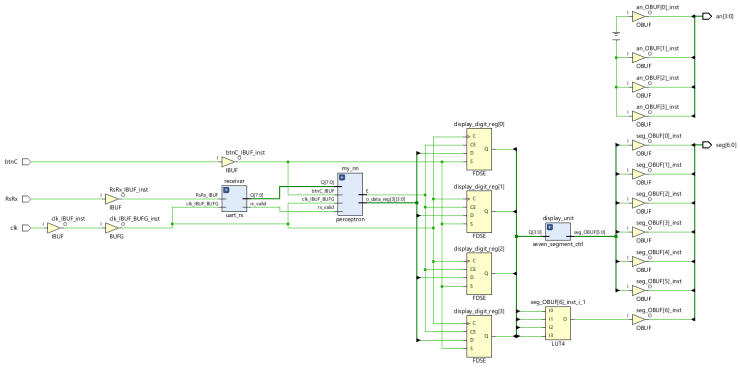


Figure 11: RTL Schematic of Top module

5.4 Resource Utilisation

The following tables provide estimated post-implementation resource figures for the Artix-7 XC7A35T target. Because all neurons are instantiated in parallel, LUT usage scales with the total number of neurons and their accumulator logic.

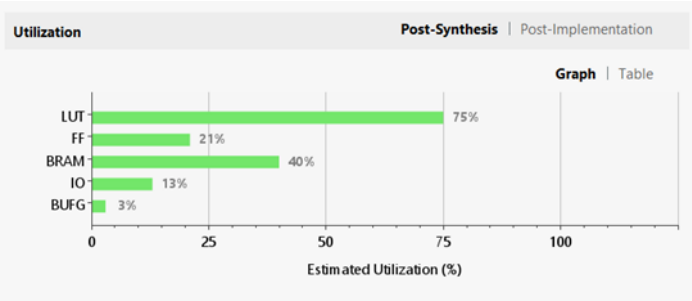


Figure 12: Post-Synthesis Resource Utilization Graph

Utilization — Post-Synthesis | Post-Implementation — Graph | Table

Resource	Estimation	Available	Utilization %
LUT	15560	20800	74.81
FF	8535	41600	20.52
BRAM	20	50	40.00
IO	14	106	13.21
BUFG	1	32	3.13

Figure 13: Post-Synthesis Resource Utilization Table

The high DSP demand is primarily due to the 130 parallel neurons (40+40+20+20+10), each instantiating one 8-bit signed multiplier. Vivado may map some multiplications to LUT-based logic when DSP resources are exhausted, increasing LUT utilisation but not affecting functional correctness.

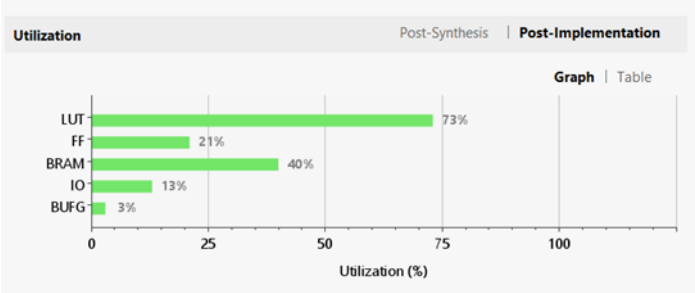
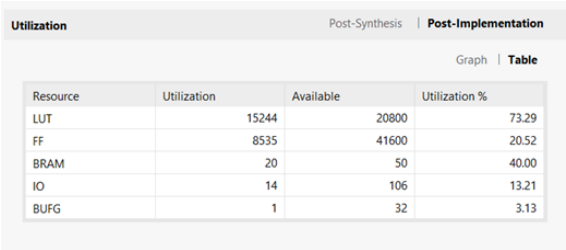


Figure 14: Post-Implementation Resource Utilization Graph



Resource	Utilization	Available	Utilization %
LUT	15244	20800	73.29
FF	8535	41600	20.52
BRAM	20	50	40.00
IO	14	106	13.21
BUFG	1	32	3.13

Figure 15: Post-Implementation Resource Utilization Table

5.5 *Timing*

The critical path runs through the neuron multiplier and accumulator. At 100 MHz (10 ns clock period), the design closes timing comfortably owing to the fully registered pipeline: input registration, multiplication, accumulation, and output registration each occur in separate clock stages with no combinational paths spanning more than one stage.

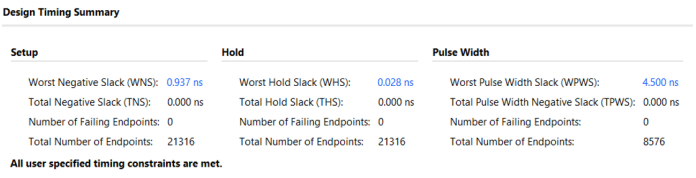


Figure 16: Timing Summary

Maximum Frequency of operation **(Fmax) = 110.33 MHz**

5.6 *Area Report*

The synthesized design of the perceptron using the ReLU activation function was analyzed to estimate hardware resource utilization. The area report provides detailed information about the number of cells used and their distribution across different logic elements.

The results indicate that the design is efficient and well-optimized for hardware implementation. The area report further demonstrates that the design is well-suited for FPGA realization, achieving a good balance between resource utilization and computational efficiency.

It is important to note that the synthesis results were obtained using a 45 nm open standard cell library, which provides a realistic estimation of silicon area and enhances the accuracy of the reported hardware metrics.

```
Number of public wire bits:  320808
Number of memories:               0
Number of memory bits:            0
Number of processes:              0
Number of cells:             173683
  AND2_X1                      9059
  AND3_X1                      4239
  AND4_X1                      2135
  AOI211_X1                    3440
  AOI21_X1                    20714
  AOI221_X1                    1312
  AOI222_X1                      65
  AOI22_X1                     3163
  DFF_X1                      10786
  INV_X1                       6836
  MUX2_X1                       688
  NAND2_X1                    19370
  NAND3_X1                     4056
  NAND4_X1                     1548
  NOR2_X1                     14784
  NOR3_X1                      3924
  NOR4_X1                      1411
  OAI211_X1                    5494
  OAI21_X1                    23833
  OAI221_X1                    1651
  OAI222_X1                       7
  OAI22_X1                     1570
  OAI33_X1                      293
  OR2_X1                       3632
  OR3_X1                       3931
  OR4_X1                        400
  XNOR2_X1                    15585
  XOR2_X1                      9757

  Chip area for top module '\perceptron': 233237.046000

Warnings: 173953 unique messages, 174017 total
End of script. Logfile hash: 8c56322a34
CPU: user 451.18s system 4.55s, MEM: 1038.12 MB total, 1014.45 MB resident
Yosys 0.9 (git sha1 1979e0b)
Time spent: 20% 17x opt_clean (90 sec), 19% 19x opt_merge (89 sec), ...
```

Figure 17: Area Report

5.7 *Power Report*

The power analysis from the implemented netlist shows the distribution of dynamic and static power across different components.

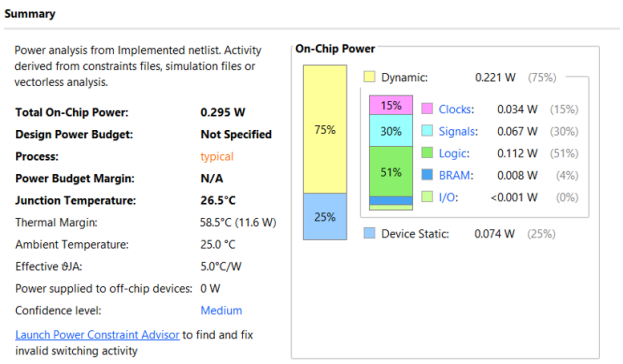


Figure 18: Power Report

5.8 *Hardware Demo on Basys 3*

In the physical demonstration, a custom Python-based GUI is used to draw a digit, the drawn image is then serialized and transmitted pixel-by-pixel (784 bytes) over a USB-UART interface at 115,200 baud. The Basys 3

FPGA receives the data, performs inference, and displays the predicted digit on the seven-segment display within approximately 70 ms, with latency dominated by UART transmission. The centre button (`btnC`) resets the display to a blank state in preparation for the next input.

VI Conclusion

This project successfully implemented a five-layer fully-connected neural network for MNIST digit recognition entirely in synthesisable Verilog on the Digilent Basys 3 FPGA board. The key contributions and findings are:

1. **Parallel-neuron architecture:** Instantiating all neurons in a layer simultaneously reduces per-layer latency from $O(N \times M)$ to $O(M)$ clock cycles, providing a linear speedup proportional to the number of neurons.
2. **Pipelined FSM-based inter-layer serialisation:** The IDLE/SEND two-state FSM converts the wide parallel output of each layer to a serial stream consumed by the next layer with minimal latency overhead and natural pipeline overlap.
3. **Fixed-point quantisation with 32-bit accumulation:** Uses different formats (Q1.7 for inputs, Q4.4 for weights, Q5.3 for biases) to balance precision and dynamic range while reducing hardware cost. The wide accumulator is essential for preventing overflow across 784 additions; the final ReLU stage requantises the result to 8 bits with saturation.
4. **Pre-trained weight initialisation:** Using `$readmemb` to load `.mif` files eliminates on-chip training logic, keeps the design compact, and enables rapid iteration by simply replacing the weight files.
5. **End-to-end hardware system:** The design integrates UART reception, multi-layer inference, argmax selection, and seven-segment display output into a single, fully self-contained FPGA system with no external processor.

The primary throughput bottleneck is the UART input link at 115,200 baud. Future work could replace UART with SPI or a parallel interface to achieve near-real-time inference on video streams, extend the architecture to deeper networks using time-multiplexed neuron reuse to reduce resource consumption, or explore post-training quantisation to even fewer bits (4-bit or binary) for ultra-low-power deployments.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[2] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, *Efficient Processing of Deep Neural Networks: A Tutorial and Survey*, Proc. IEEE, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[3] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, *Optimizing FPGA-Based Accelerator Design for Deep Convolutional Neural Networks*, in *Proc. ACM/SIGDA FPGA*, Feb. 2015, pp. 161–170.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. Available: http://www.deeplearningbook.org

[5] Xilinx Inc., *Vivado Design Suite User Guide: Synthesis (UG901)*, Version 2023.1, 2023.

[6] MNIST Handwritten Digit Database, http://yann.lecun.com/exdb/mnist/, accessed Apr. 2026.

[7] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, 2023. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual